

Checkers – Java

Rapport final

RANDRIAMIHAJA Latiana
RAMANANAHERISOA Fy-Tommy
HOAREAU Faniry
ATWI Daniel
RAZAFINDRAKOTO Sarah

8 avril 2026

Introduction

1 Contexte et objectifs du projet

Le projet a pour objectif de concevoir et développer un logiciel permettant de jouer au jeu de dames conformément aux règles officielles. Le programme doit permettre à deux joueurs de s'affronter sur un damier, soit en mode local, soit à distance via un réseau.

Le logiciel garantit le respect strict des règles du jeu tout en proposant une expérience utilisateur claire, robuste et extensible. Il s'inscrit dans une démarche de développement logiciel complète, intégrant la qualité du code, la documentation, les tests et les performances.

1.1 Variantes supportées

Le jeu de dames se décline en plusieurs variantes internationales, que le programme supporte via une option de configuration du plateau. Le style anglo-saxon, appelé Draughts, se joue sur un plateau de 8×8 , chaque joueur disposant de 12 pions. Le jeu de dames international adopte quant à lui un plateau de 10×10 , avec 20 pions par joueur. Enfin, les dames canadiennes étendent encore la surface de jeu avec un plateau de 12×12 , portant le nombre de pions à 30 par joueur.

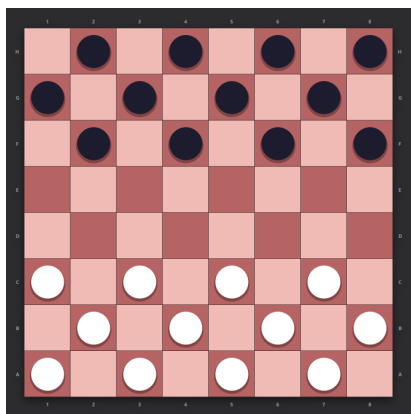


FIGURE 1 – Plateau 8×8

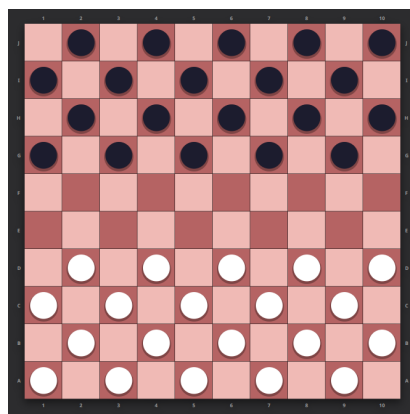


FIGURE 2 – Plateau 10×10

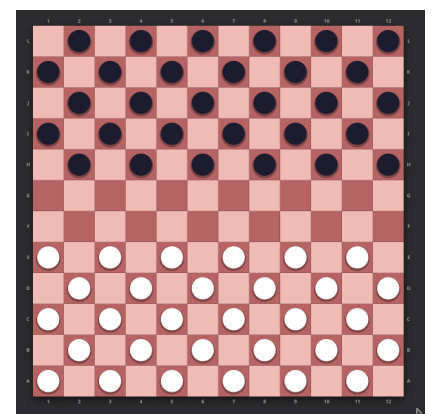


FIGURE 3 – Plateau 12×12

1.2 Règles du jeu

1.2.1 Mise en place et déroulement

Le damier est orienté avec une case foncée en bas à gauche. Les pions se placent exclusivement sur les cases foncées des quatre premières rangées de chaque camp. Les joueurs jouent chacun à leur tour ; les blancs commencent toujours.

1.2.2 Déplacements

Un pion se déplace d'une case vers l'avant en diagonale. Lorsqu'il atteint la dernière rangée adverse et s'y arrête, il est promu en dame, ce qui se matérialise visuellement par l'ajout d'un point jaune. La dame bénéficie alors d'une mobilité bien supérieure : elle peut parcourir librement une diagonale entière, d'autant de cases qu'elle le souhaite, aussi bien vers l'avant que vers l'arrière.

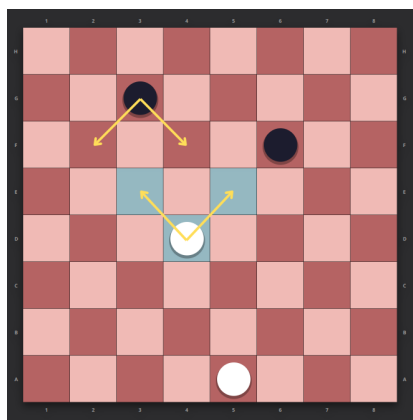


FIGURE 4 – Déplacement d'un pion

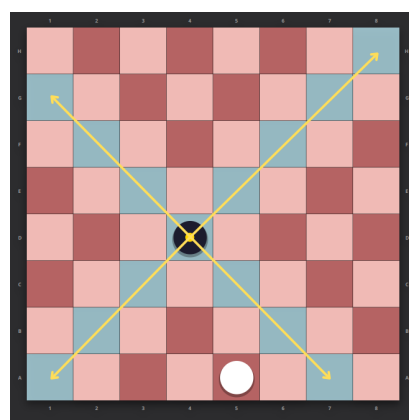


FIGURE 5 – Déplacement d'une dame

1.2.3 La prise par un pion

Un pion capture un pion adverse en sautant par-dessus lui pour se poser sur la case vide située immédiatement derrière. La prise peut s'effectuer vers l'avant comme vers l'arrière.

La prise est obligatoire. De plus, si après une capture le pion se retrouve à nouveau en position de prise, il doit continuer jusqu'à épuisement des captures possibles (rafle). Les pions capturés sont retirés du plateau uniquement à l'issue complète de la rafle, et non au fur et à mesure.

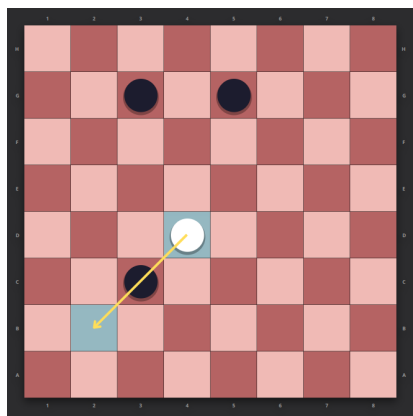


FIGURE 6 – Prise d'un pion

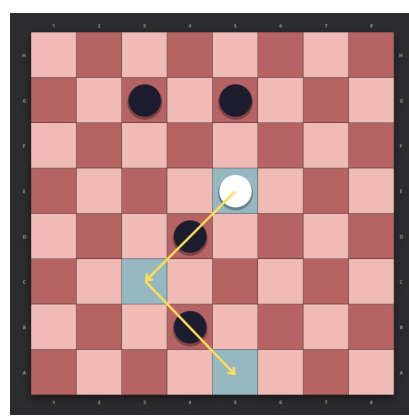


FIGURE 7 – Rafle

1.2.4 La prise majoritaire

Lorsque plusieurs séquences de capture sont possibles, le joueur est tenu de choisir celle qui capture le plus grand nombre de pièces, quel que soit leur rang. Ainsi, une séquence permettant de prendre deux pions est prioritaire sur la prise d'une dame isolée.

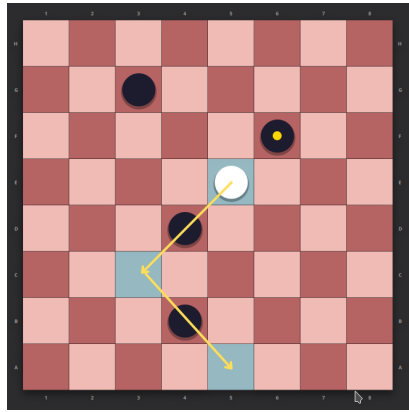


FIGURE 8 – Prise majoritaire : 2 pions vs 1 dame

1.2.5 La prise par la dame

Grâce à son amplitude de mouvement, la dame dispose de capacités de capture étendues. Elle doit prendre tout pion situé sur sa diagonale dès lors qu'une case libre est disponible immédiatement après. On ne peut passer qu'une seule fois sur un même pion, mais on peut traverser deux fois la même case.

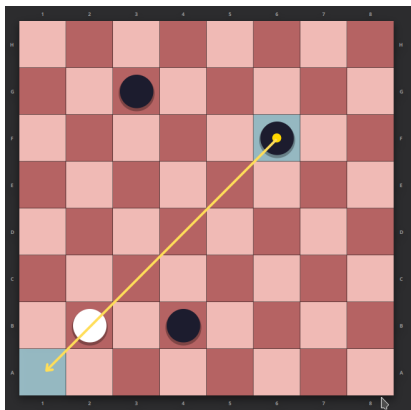


FIGURE 9 – Prise par la dame

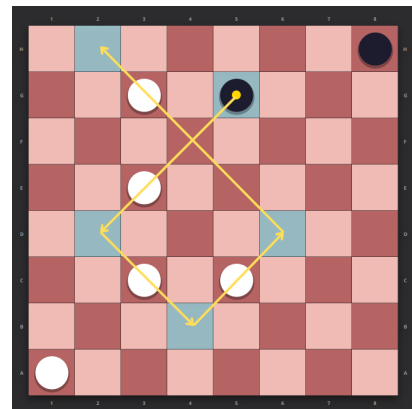


FIGURE 10 – Prise par la dame avec passage sur une même case

1.3 Conditions de fin de partie

Une partie prend fin dans l'un des cas suivants :

Victoire par capture. Un joueur remporte la partie en capturant l'intégralité des pièces adverses.

Victoire par blocage. Si un joueur ne peut effectuer aucun déplacement alors qu'il lui reste des pièces, il perd immédiatement la partie.

Nulle par accord de position. La partie est déclarée nulle si la même position se reproduit trois fois au cours de la partie (règle de la répétition triple).

Nulle par absence de progrès. Si aucune capture ni aucun mouvement de pion n'est effectué pendant 25 tours consécutifs (soit 50 demi-coups), la partie est déclarée nulle.

Nulle en fin de partie. Lorsqu'un joueur ne dispose plus que d'une seule pièce, si aucune issue décisive n'est atteinte en 16 tours (soit 32 demi-coups), la partie est déclarée nulle faute de matériel suffisant.

Nulle par blocage mutuel. Dans le cas exceptionnel où aucun des deux joueurs ne peut effectuer de mouvement, la partie est également déclarée nulle.

1.4 Versions existantes

Dans le cadre de notre phase d'analyse, nous avons étudié deux projets open-source de jeu de dames afin d'identifier des approches architecturales et algorithmiques pertinentes pour notre développement.

AwsDame

AwsDame [2] est une plateforme de jeu de dames multijoueur en temps réel développée dans un environnement JavaScript/Node.js. Bien que la pile technologique diffère de la nôtre (Java), son architecture réseau orientée événements offre des points de comparaison instructifs.

En particulier, ce projet adopte une logique de *matchmaking automatique* entre joueurs connectés, une idée que nous avons reprise dans notre propre serveur TCP. Notre classe `GameServer` implémente en effet une méthode `tryAutoStart()` qui détecte automatiquement la disponibilité de deux joueurs inactifs et déclenche une session sans intervention manuelle :

```
// GameServer.java -- démarrage automatique d'une partie
List<PlayerSession> idlePlayers = registry.getAllPlayers().stream()
    .filter(PlayerSession::isIdle)
    .collect(Collectors.toList());

if (idlePlayers.size() >= 2) {
    List<PlayerSession> pair = idlePlayers.subList(0, 2);
    GameSession session = registry.createSession(pair, sessionController);
    // notification GAME_START envoyée aux deux joueurs
}
```

Contrairement à *AwsDame* qui s'appuie sur WebSockets, notre solution repose sur des sockets TCP bruts avec un protocole textuel simple (REGISTER, MOVE, GAME_OVER, etc.), ce qui facilite les tests en ligne de commande et l'intégration avec un client Java natif.

Raven Checkers

Raven Checkers [1] est un programme de dames open-source développé en Python, conçu à des fins pédagogiques pour l'étude des tactiques de jeu et des algorithmes d'intelligence artificielle. Il couple la fonction d'évaluation de *Simple Checkers* aux algorithmes de recherche du projet AIMA [3].

L'analyse de ce projet a principalement influencé notre implémentation des algorithmes d'IA. *Raven Checkers* propose un MinMax classique sans gestion du temps, ce qui le rend peu adapté à des contraintes de jeu réel. Notre implémentation y répond de trois façons :

1. **Approfondissement itératif.** Plutôt que de fixer une profondeur statique, nos classes `MinMax` et `MinMaxAlphaBeta` explorent l'arbre niveau par niveau, de la profondeur 1 jusqu'à `maxDepth`, et conservent uniquement le résultat du dernier niveau entièrement exploré. Si le budget temps expire en cours de niveau, l'exploration partielle est abandonnée via une exception interne `TimeExceededException`, garantissant qu'aucun résultat incomplet n'est retenu.
2. **Élagage alpha-bêta.** La classe `MinMaxAlphaBeta` maintient deux bornes α (meilleur score garanti pour le maximisant) et β (meilleur score garanti pour le minimisant). Dès que $\beta \leq \alpha$, les branches restantes sont élaguées, réduisant significativement le nombre de nœuds évalués par rapport au MinMax pur.
3. **MCTS avec mode de sélection configurable.** Au-delà du MinMax, notre classe `Mcts` implémente la méthode de Monte Carlo Tree Search, avec un mode de sélection paramétrable (UCT ou ML) défini par l'énumération `SelectionMode`. Cette flexibilité est absente de *Raven Checkers*, qui ne propose qu'un seul algorithme de recherche.

La fabrique `Ai.buildAi(Configuration)` centralise le choix de l'algorithme à l'exécution :

```
// Ai.java -- sélection de l'algorithme via la configuration
switch (aiMode.toLowerCase()) {
    case "minimax" -> return new MinMax(depth, timeMs);
    case "alphabeta" -> return new MinMaxAlphaBeta(depth, timeMs);
    case "mcts" -> {
        Mcts mcts = new Mcts(depth, timeMs);
        mcts.setSelectionMode(cfg.getSelectionMode());
        return mcts;
    }
}
```

Bilan comparatif

Le tableau 1 synthétise les principales différences entre les deux projets analysés et notre propre implémentation.

L'étude de ces deux projets nous a permis de valider nos choix de conception et d'identifier les lacunes à combler : gestion du temps dans la recherche adversariale, richesse algorithmique de l'IA, et protocole réseau adapté à un client Java interactif.

TABLE 1 – Comparaison des projets existants et de notre implémentation

Critère	AwsDame	Raven Checkers	Notre projet
Langage	JavaScript	Python	Java
Transport réseau	WebSocket	—	TCP (sockets)
Matchmaking automatique	Oui	—	Oui
Suivi des scores	—	—	Oui (W/L/D)
MinMax	—	Oui	Oui
Élagage alpha-bêta	—	Non	Oui
Approfondissement itératif	—	Non	Oui
MCTS	—	Non	Oui
Contrainte de temps pour l'IA	—	Non	Oui (configurable)

2 Algorithme et Structure de données

2.1 Bitboard

Nous utilisons des bitboards pour représenter l'état du plateau. Cette approche permet de représenter l'état du plateau sous forme d'entiers primitifs et de calculer les coups de manière globale via des opérations bit à bit, offrant des performances bien supérieures aux matrices 2D classiques.

2.1.1 Représentation et Numérotation des cases

Sur les 100 cases d'un plateau standard (10×10), seules les 50 cases noires sont utilisées. Nous utilisons la **notation de Manoury** pour numéroter ces cases actives de 0 à 49, de bas en haut et de gauche à droite.

L'état complet du jeu est encodé sur quelques entiers de 64 bits (`long` en Java), où chaque bit correspond à une case active (1 si la case est occupée, 0 si elle est vide). Cela nous évite d'écrire des formules mathématiques complexes pour définir l'état du plateau : tout repose sur l'indexation binaire.

	.	45	.	46	.	47	.	48	.	49
40	.	41	.	42	.	43	.	44	.	
	.	35	.	36	.	37	.	38	.	39
30	.	31	.	32	.	33	.	34	.	
	.	25	.	26	.	27	.	28	.	29
20	.	21	.	22	.	23	.	24	.	
	.	15	.	16	.	17	.	18	.	19
10	.	11	.	12	.	13	.	14	.	
	.	5	.	6	.	7	.	8	.	9
0	.	1	.	2	.	3	.	4	.	

2.1.2 Gain de mémoire chiffré

L'intérêt premier du bitboard est la compression des données. Comparons cette approche avec une structure de données "classique" comme un tableau à deux dimensions en Java (`int[10][10]`). Un tableau 2D classique nécessite 100 cases, chacune stockant un entier de 32 bits, soit un total de 3200 bits pour représenter l'état du plateau. En revanche, avec les bitboards, nous représentons l'ensemble du plateau avec seulement 4 variables de type `long` (64 bits chacune) : une pour les pions blancs, une pour les pions noirs, un pour les dames blanches et un pour les dames noires. Cela représente seulement 256 bits au total.

Le gain en mémoire vive est d'un facteur 12, ce qui accélère drastiquement les copies de plateaux lors de la recherche de l'IA.

2.1.3 Application des opérations binaires

L'avantage majeur du bitboard n'est pas seulement le stockage, mais la manipulation du plateau via le processeur. Au lieu de parcourir le plateau case par case, nous utilisons :

- **ET logique** ($\wedge$) : Pour détecter les collisions (ex : croiser un masque de déplacement avec le bitboard des pièces adverses).
- **OU logique** ($\vee$) : Pour fusionner les bitboards (ex : Blancs $\vee$ Noirs) et obtenir la carte de toutes les cases occupées.

- **NON logique** ($\neg$) : Pour inverser la carte d'occupation et isoler instantanément toutes les cases vides du damier.
- **Décalage binaire** ($\ll$, $\gg$) : Pour traduire toutes les pièces d'un joueur dans une direction donnée en une seule instruction d'horloge.

2.2 Algorithmes de manipulation

L'intérêt de la section précédente prend tout son sens ici : la génération des coups légaux repose sur la translation temporelle des bits.

TABLE 2 – Comparaison des décalages binaires selon la taille du plateau

Taille Plateau	8x8 (32 bits)		10x10 (50 bits)		12x12 (72 bits)	
Transition Ligne	P → I	I → P	P → I	I → P	P → I	I → P
Haut-Gauche	+3	+4	+4	+5	+5	+6
Haut-Droite	+4	+5	+5	+6	+6	+7
Bas-Gauche	-5	-4	-6	-5	-7	-6
Bas-Droite	-4	-3	-5	-4	-6	-5

2.2.1 Déplacement et prise par un pion

Lorsqu'un pion se déplace, on décale l'ensemble de son bitboard d'une direction donnée. Si le bit "atterrit" sur une position valide (une case vide, vérifiée via une opération $\wedge$ sur le bitboard inversé $\neg(\text{Occupées})$), le coup est légal.

Exemple concret de prise : Pour qu'une capture soit possible, un pion doit sauter par-dessus un pion adverse et atterrir sur une case vide. Sur un plateau 10×10 , supposons un pion blanc en case 32 (ligne paire). S'il veut capturer un pion noir situé en haut à gauche (case 27, ligne impaire), le décalage pour atteindre le pion noir est de +5. Pour atterrir derrière ce pion, le saut continue vers la case 21 (ligne paire), nécessitant un nouveau décalage de +6.

Le décalage total pour cette prise est donc constant : $5 + 6 = 11$ bits. L'opération binaire simplifiée de cette validation ressemble à cela :

$$\text{CibleSaut} = (\text{PionBlanc} \gg 5) \wedge \text{PionNoir}$$

$$\text{AtterrissageValide} = (\text{CibleSaut} \gg 6) \wedge \neg(\text{Occupées})$$

2.2.2 Déplacement et prise par une dame

Contrairement au pion, la dame peut parcourir un nombre illimité de cases vides sur une diagonale. D'un point de vue binaire, un simple décalage ne suffit pas : il faut propager le bit sur toute la diagonale jusqu'à rencontrer un obstacle. C'est la technique du *sliding piece generation*.

Voici le pseudo-code illustrant la génération des coups possibles pour une dame :

```
Fonction GenererCoupsDame(bitboard_dame, direction):
    coups_possibles = 0
    // Premier decalage dans la direction souhaitee
    rayons = decaler(bitboard_dame, direction)

    // Tant qu'on se deplace sur des cases vides
    Tant que rayons != 0:
        // On valide les cases vides rencontrees comme coups possibles
        coups_possibles = coups_possibles OU (rayons ET cases_vides)

        // On continue la propagation uniquement depuis les cases vides
        // La propagation s'arrete si on touche une piece ou le bord
        rayons = decaler(rayons ET cases_vides, direction)

    Retourner coups_possibles
```

3 Architecture du projet

3.1 Le Moteur de Jeu

L'architecture de notre application repose sur le patron Modèle-Vue-Contrôleur (MVC). Cette section détaille la conception de notre moteur de jeu (qui incarne le Modèle) et la façon dont il interagit avec le reste du programme pour assurer le bon déroulement d'une partie.

3.1.1 Rôle et responsabilités

Le moteur de jeu, regroupé dans le package `model`, a pour unique mission de simuler et de faire tourner la partie de dames. Il se concentre purement sur la logique mathématique et les règles officielles : il ne gère absolument pas les clics de souris, la saisie clavier ou l’affichage graphique.

Ses responsabilités sont réparties entre deux classes majeures. D’un côté, la classe `Board` agit comme le moteur physique du damier : elle manipule les pièces, valide si un coup respecte la géométrie du plateau et applique les règles de prises obligatoires. De l’autre, la classe `GameCheckers` incarne l’arbitre de la partie. C’est elle qui maintient l’état global du système. Elle garde la trace du joueur actif, stocke l’historique des mouvements pour permettre les retours en arrière (undo/redo), et détecte les conditions de fin de partie (victoire, blocage, ou match nul par répétition).

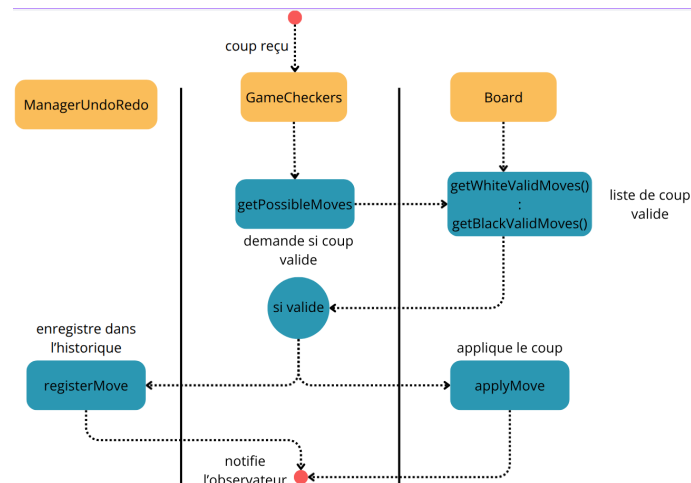


FIGURE 11 – Architecture du Modèle

3.1.2 Interface de communication (API)

Pour que le reste de l’application puisse interagir avec ce moteur, `GameCheckers` expose une interface de communication claire basée sur des actions d’entrée et des notifications de sortie.

En entrée, l’interface graphique ou textuelle ne modifie jamais le plateau directement. Elle passe par le `GameController` qui interroge d’abord le moteur via `getPossibleMoves(player)` pour s’assurer qu’une action est légale, puis lui demande de l’exécuter avec la méthode `applyMove(from, to)`.

En sortie, le moteur dicte le rythme grâce au patron de conception Observateur. Au lieu de forcer l’interface graphique à vérifier constamment si quelque chose a changé, le moteur travaille en silence et émet un signal de mise à jour via `notifyObservers()` uniquement lorsqu’un mouvement valide vient de modifier l’état du jeu.

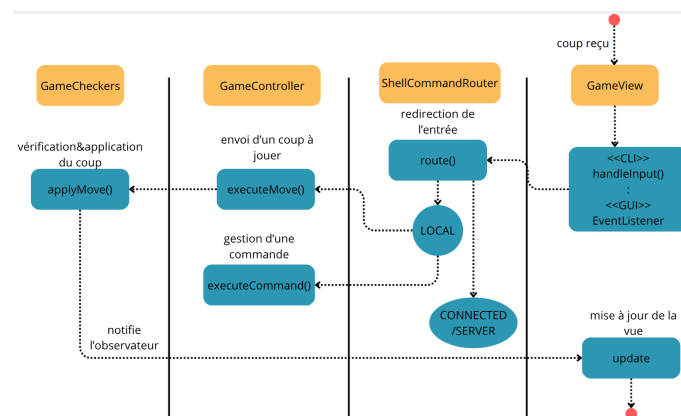


FIGURE 12 – Interaction entre le moteur et la vue via le contrôleur

Note : La bulle « CONNECTED/SERVER » visible sur ce schéma représente le point d’entrée vers les fonctionnalités réseaux. Le fonctionnement détaillé de ce module (protocole TCP, sockets et gestion du serveur) est traité spécifiquement dans la section 3.3.

3.1.3 Choix techniques et boucle de contrôle

D'un point de vue technique, la représentation du plateau ne repose pas sur un simple tableau à deux dimensions, mais sur des *bitboards* (comme détaillé dans la section algorithmique 2). Ce choix garantit un accès et une manipulation en temps $O(1)$ via des opérations binaires.

Concernant la dynamique, le jeu s'exécute au tour par tour, géré par une machine à états simple définie par l'énumération `State` (`IN_GAME`, `PAUSE`, `FINISHED`). L'évolution du système lors de l'application d'un coup suit toujours la même séquence logique stricte :

```
// Extrait simplifié de GameCheckers.java : évolution de l'état du système
public void applyMove(String from, String to, boolean isManoury) {
    // 1. Délégation de la physique au plateau
    board.applyMove(move);

    // 2. Mise à jour de l'état global (historique et tour de jeu)
    managerUndoRedo.registerMove(currentColor, move);
    this.isWhiteTurn = !this.isWhiteTurn;

    // 3. Signal de fin d'opération à l'interface
    notifyObservers();
}
```

Une fois le coup appliqué et la vue rafraîchie, le `GameController` vérifie si la partie est terminée. Si le jeu continue, il oriente le flux vers une attente de saisie utilisateur, ou lance immédiatement la réflexion de l'algorithme si le joueur suivant est une intelligence artificielle.

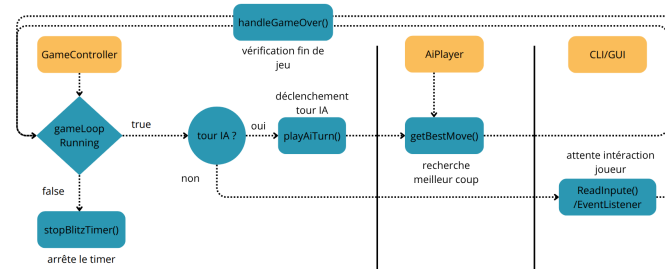


FIGURE 13 – Boucle de contrôle du jeu

3.2 Module affichage

Le module d'affichage correspond à la couche Vue de notre architecture MVC. Dans la pratique, il repose sur une base commune, `GameView`, puis sur trois implémentations adaptées au contexte d'exécution : `CommandLineInterface` pour le terminal, `GraphicalUserInterface` pour JavaFX, et `HeadlessView` côté serveur lorsque l'on veut exécuter une partie sans affichage. Ce découpage a été choisi très tôt pour éviter de mélanger logique d'interaction, rendu visuel et règles du jeu.

Le point important est que le contrôleur ne connaît que l'abstraction `GameView`. Quand un joueur agit, la vue transforme cette action en appel contrôleur (commande ou déplacement), puis le moteur met à jour l'état de jeu. La vue n'écrit jamais directement dans le modèle. Une fois le coup traité, le mécanisme d'observation notifie les vues, qui se contentent de se rafraîchir. Ce fonctionnement paraît simple, mais c'est ce qui nous a permis de garder exactement les mêmes règles en CLI, en GUI et en mode headless.

En CLI, nous avons utilisé `JLine` pour ne pas nous limiter à un `Scanner` minimaliste. On bénéficie ainsi d'une saisie plus robuste (édition de ligne, complétion, gestion propre des interruptions) et d'un comportement plus stable pendant les tests. Nous avons aussi séparé volontairement la lecture d'entrée du routage des commandes : `CommandLineInterface` lit et affiche, tandis que `ShellCommandRouter` décide quoi faire de la commande. Cette séparation nous a évité une classe trop chargée et a rendu les tests de parsing beaucoup plus simples.

En GUI, la même logique a été conservée, avec une organisation en sous-vues (`MainView`, `PlayView`, `BoardView`, `LogView`) pour garder du code lisible. Le principal choix technique ici concerne les threads : une mise à jour visuelle doit toujours passer par `Platform.runLater(...)`. Sans cela, les notifications venant du timer blitz ou d'un calcul d'IA peuvent provoquer des erreurs JavaFX difficiles à reproduire. Nous avons également lié la taille des cases à la taille de la fenêtre via des bindings, pour conserver un damier correct sur les trois formats de plateau pris en charge.

Avec le recul, ce module fonctionne surtout parce qu'il reste discipliné : l'affichage reste un adaptateur, pas un second moteur de jeu. C'est ce qui nous permet d'ajouter des fonctionnalités d'interface sans toucher aux règles, et d'exécuter les mêmes scénarios de jeu dans des environnements très différents.

3.2.1 Architecture interne de la GUI

L'implémentation de la GUI repose sur javafx et une décomposition en plusieurs composants spécialisés :

GraphicalUserInterface et le pattern Observer La classe `GraphicalUserInterface` étend `GameView`, ce qui lui permet de s'intégrer directement au pattern Observer déjà utilisé par le projet. Quand le modèle change, la GUI reçoit l'information via la méthode `update (GameCheckers)` et redirige la mise à jour vers les composants visuels.

Le point central ici est la gestion du thread JavaFX. Comme JavaFX impose que toute modification de l'interface passe par son thread dédié (Application Thread), les mises à jour sont dispatchées via `Platform.runLater (...)` :

```
@Override
public void update (GameCheckers game) {
    Platform.runLater(() -> {
        if (mainView != null) {
            mainView.update (game);
        }
    });
}
```

Ce pattern de thread est **central** : sans lui, toute mise à jour du modèle provenant d'un autre thread provoque une exception `IllegalStateException`. La classe gère aussi l'initialisation de la fenêtre principale, l'application de la feuille de style CSS et la fermeture contrôlée avec vérification des sauvegardes avant quitter.

MainView et organisation visuelle La fenêtre principale est construite par `MainView`, qui hérite de `BorderPane`. Ce choix permet de séparer proprement l'interface en trois zones : le menu en haut, le jeu au centre et la barre d'actions en bas.

`MainView` ne redessine pas le plateau lui-même ; ce rôle est délégué à `PlayView`, ce qui évite de concentrer la logique d'affichage dans une seule classe. Cette délégation renforce la maintenabilité et facilite les tests unitaires de chaque composant.

BoardView et rendu interactif `BoardView` est le composant qui affiche réellement le damier, les pièces, les sélections, les coups suggérés et les mouvements possibles. Le composant repose sur un `GridPane`, reconstruit visuellement à chaque redraw. L'interaction suit un schéma à deux clics : au premier clic, la pièce est sélectionnée et surlignée en bleu ; au second clic, la destination est envoyée au contrôleur pour exécution du coup.

Un point crucial est que la GUI **ne recalcule pas** les règles de dames. Elle demande directement au modèle quels coups sont légaux, puis ne garde que ceux qui concernent la pièce sélectionnée :

```
List<Move> possibleMoves = game.getPossibleMoves (game.getCurrentPlayer());

for (Move m : possibleMoves) {
    if (m.getFrom() == selectedIndex) {
        validDestinationSquares.add (board.indexToSquare (m.getTo()));
    }
}
```

L'interface ne décide pas ce qui est légal ou non ; elle se contente d'afficher proprement ce que le moteur autorise. Ce choix architectural évite de dupliquer la logique métier dans la vue et garantit la cohérence globale : une règle modifiée dans le moteur s'applique immédiatement sans changement de code GUI.

Le drag and drop a également été implémenté. Quand l'utilisateur commence à faire glisser une pièce, la case de départ est stockée dans le `Dragboard` JavaFX. Au dépôt, la vue reconstruit simplement le déplacement et l'envoie au contrôleur sans validation locale. Encore une fois, la vue gère l'interaction (UX), mais pas les règles.

ConfigDialog et interactions complexes Le dialogue de configuration `ConfigDialog` va au-delà d'une simple présentation de cases à cocher. Les composants réagissent entre eux : si le mode blitz est désactivé, le spinner de temps l'est aussi. Si aucune IA n'est active, les options liées aux IA sont désactivées automatiquement. Ces dépendances sont gérées via des listeners JavaFX sur les propriétés des contrôles.

Plus important encore, le dialogue reconstruit un objet `Configuration` exploitable directement par le moteur. Il ne renvoie pas juste des valeurs d'interface brutes : il normalise les choix, convertit les unités (par exemple, les secondes en millisecondes pour l'IA) et prépare les bons formats.

Tests de la GUI Exemple : `ConfigDialog`. Les tests utilisent JUnit 5, TestFX (framework pour tester JavaFX) et Mockito (pour simuler le contrôleur). Les composants JavaFX sont créés sur le bon thread à l'aide de `Platform.runLater(...)`, puis les interactions utilisateur sont simulées avec `FxRobot`.

Par exemple, un test vérifie que cocher ou décocher le mode blitz active ou désactive correctement le spinner de temps :

```
robot.interact(() -> blitzCheck.setSelected(false));
assertTrue(timeSpinner.isDisabled());

robot.interact(() -> blitzCheck.setSelected(true));
assertFalse(timeSpinner.isDisabled());
```

Cette approche garantit que le comportement du dialogue reste stable lors des refactorisations futures et valide que les interactions entre composants restent cohérentes.

3.3 Module Réseau

La couche réseau de l'application s'appuie sur une architecture client-serveur TCP, basant principalement sur le package `fr.ubordeaux.pdp.server`. La figure 14 en présente les composants principaux ainsi que leurs interactions.

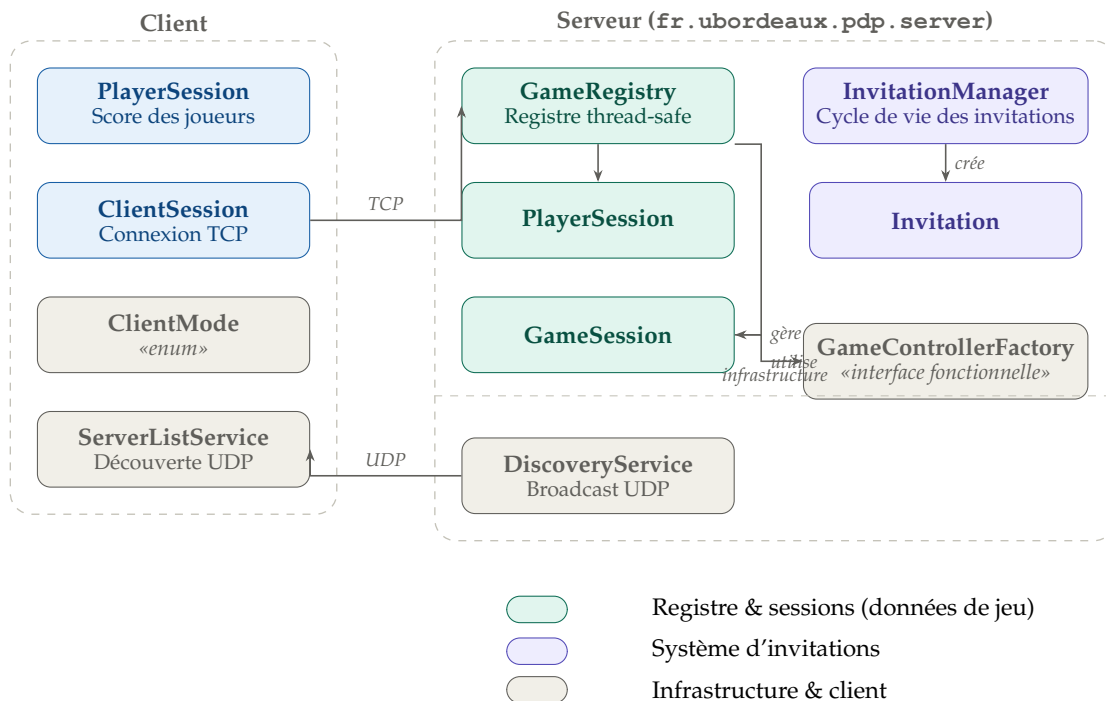


FIGURE 14 – Architecture structurelle de la couche réseau

Côté client. Le `ShellCommandRouter` constitue le point d'entrée unique pour toute saisie utilisateur. Il consulte l'énumération `ClientMode` (`LOCAL`, `CONNECTED`, `SERVER`) pour aiguiller chaque commande vers le bon gestionnaire, évitant ainsi les états incohérents (par exemple, démarrer un serveur alors qu'une connexion est déjà ouverte). La `ClientSession` encapsule la socket TCP, le fil d'écoute (*listener thread*) et le protocole de *handshake* (`REGISTER / WELCOME`). Le `ServerListService` écoute les annonces UDP diffusées par `DiscoveryService` afin de présenter la liste des serveurs disponibles sur le réseau local.

Côté serveur. Le `GameRegistry` centralise, de façon thread-safe, l'ensemble des `PlayerSession` et des `GameSession` actives. Il constitue la source de vérité pour l'état de chaque joueur connecté. L'`InvitationManager` orchestre le cycle de vie complet des invitations — de leur création jusqu'à l'acceptation, au refus ou à l'expiration automatique par un fil démon (*sweeper*) — en s'appuyant sur les objets `Invitation` immuables. Enfin, la `GameControllerFactory`, interface fonctionnelle, permet d'instancier un contrôleur de jeu isolé pour chaque session, garantissant l'indépendance des états de parties concurrentes.

4 Intelligence Artificielle

4.1 SimpleEvaluator

La classe `SimpleEvaluator` peut être vue comme une balance très basique : elle compte simplement le "matériel" sur le plateau, c'est-à-dire combien de pions et de dames possède chaque joueur. Si les blancs ont plus de pièces fortes, le score monte ; si les noirs sont avantagés, le score baisse. Cette approche est facile à comprendre, rapide, mais elle ne tient pas compte de la position stratégique des pièces.

4.2 AdvancedEvaluator

La classe `AdvancedEvaluator` reprend cette idée de balance, mais ajoute une intuition naturelle du jeu de dames : un pion qui avance vers la promotion devient plus dangereux. En pratique, cette classe récompense les pions qui progressent vers la dernière rangée. Elle reste simple, mais elle joue déjà plus "intelligemment" qu'un simple comptage de pièces.

4.3 MaxEvaluator

La classe `MaxEvaluator` va encore plus loin. En plus du matériel et de l'avancement, elle valorise le contrôle du centre du damier et la mobilité (le nombre de coups légaux disponibles). Intuitivement, un joueur qui contrôle le centre et qui garde beaucoup d'options est souvent en meilleure posture. Cette évaluation est plus riche et souvent plus précise, mais elle demande davantage de calcul.

4.4 MinMax

La classe `MinMax` représente la logique "je joue un coup, puis j'anticipe la meilleure réponse adverse". L'algorithme explore un arbre de possibilités en alternant les deux joueurs : il suppose que l'adversaire joue bien, et il cherche le coup qui reste le plus favorable malgré cela. Cette méthode donne des décisions solides, mais son coût augmente rapidement quand on approfondit la recherche.

4.5 MinMaxAlphaBeta

La classe `MinMaxAlphaBeta` suit exactement la même philosophie que `MinMax`, mais elle évite de calculer des branches qui ne pourront de toute façon pas changer la décision finale. On peut l'imaginer comme une lecture intelligente d'un arbre où l'on saute les chapitres déjà "perdus d'avance". Le résultat reste de même qualité, mais le temps de calcul peut être fortement réduit.

4.6 IterativeDeepening

La classe `IterativeDeepening` applique une stratégie très pratique en contexte réel : elle commence par une recherche peu profonde (rapide), puis recommence avec une profondeur plus grande, et ainsi de suite, jusqu'à épuisement du temps autorisé. L'avantage est qu'on dispose toujours d'un coup jouable, même si la recherche complète la plus profonde n'a pas le temps de se terminer.

5 Complexité

5.1 Complexité de l'IA

5.1.1 Hypothèses et notations

Dans cette section, nous analysons la complexité temporelle des méthodes principales des classes IA fournies. La variable n désigne le nombre de cases actives du plateau (typiquement 32, 50 ou 72), b représente le facteur de branchement moyen (nombre de coups légaux par position), et d la profondeur maximale de recherche. Nous distinguons la complexité *propre* d'une méthode de recherche (nombre de nœuds explorés) du coût réel, qui inclut aussi la génération des coups par le moteur via `getWhiteValidMoves()` et `getBlackValidMoves()`.

5.1.2 Complexité des fonctions d'évaluation

SimpleEvaluator La méthode `evaluate(Board)` réalise un parcours linéaire de l'ensemble des cases actives, avec uniquement des tests bit-à-bit en temps constant à chaque itération. Sa complexité est donc $O(n)$.

AdvancedEvaluator La méthode `evaluate(Board)` suit le même schéma que `SimpleEvaluator` : un balayage linéaire des cases et un calcul de bonus d'avancement en temps constant par case. La complexité reste donc $O(n)$.

MaxEvaluator La méthode `evaluate(Board)` combine trois contributions : un parcours principal en $O(n)$, un test de centralité `isInCenter(...)` en $O(1)$ par case, puis une évaluation de mobilité dont le coût dépend des appels `getWhiteValidMoves()` et `getBlackValidMoves()`. Sa complexité globale s'écrit donc $O(n + M_w + M_b)$, où M_w et M_b représentent les coûts de génération des coups blancs et noirs. La méthode `isInCenter(int, int, int)` elle-même ne fait que des comparaisons arithmétiques constantes et s'exécute en $O(1)$.

5.1.3 Complexité des algorithmes de recherche adversariale

MinMax La méthode `calculateBestMove(...)` lance une recherche complète jusqu'à `maxDepth` et hérite donc d'une complexité asymptotique en $O(b^d)$, hors coût exact de génération des coups. La méthode `findBestMoveAtDepth(...)` itère d'abord sur les coups racines puis délègue à l'exploration récursive `minMax` / `maxMin`, ce qui conduit au même ordre de grandeur. Plus finement, `minMax(...)` et `maxMin(...)` coûtent $O(b^k)$ à profondeur restante k , soit $O(b^d)$ au pire depuis la racine. Enfin, `checkTime(long)` effectue un test booléen constant en $O(1)$.

MinMaxAlphaBeta Les méthodes `calculateBestMove(...)` et `findBestMoveAtDepth(...)` suivent la même logique que `MinMax`, mais ajoutent les bornes α et β pour couper des sous-arbres. Ainsi, `maxValue(...)` et `minValue(...)` restent en $O(b^d)$ dans le pire cas (ordre des coups défavorable), mais peuvent atteindre $O(b^{d/2})$ dans le meilleur cas théorique avec un ordre des coups optimal. La méthode `checkTime(long)` reste en $O(1)$.

IterativeDeepening La méthode `calculateBestMove(...)` enchaîne des recherches successives de profondeur $1, 2, \dots, d$ tant que le budget temps n'est pas épuisé. Son coût total théorique est

$$\sum_{k=1}^d O(b^k) = O(b^d).$$

En pratique, le surcoût par rapport à une unique recherche à profondeur d est modéré, et il est compensé par la robustesse de la stratégie : en cas d'interruption temporelle, le meilleur coup d'une profondeur entièrement terminée est déjà disponible.

5.1.4 Synthèse

TABLE 3 – Complexité détaillée, méthode par méthode, des classes IA

Classe	Méthode	Complexité	Indice dans le code
<code>SimpleEvaluator</code>	<code>evaluate(Board)</code>	$O(n)$	Une boucle unique parcourt les cases actives de 0 à <code>board.getIndexMax()-1</code> , avec tests de bits en temps constant.
<code>AdvancedEvaluator</code>	<code>evaluate(Board)</code>	$O(n)$	Même schéma que <code>SimpleEvaluator</code> : boucle linéaire + calcul arithmétique constant du bonus d'avancement.
<code>MaxEvaluator</code>	<code>evaluate(Board)</code>	$O(n + M_w + M_b)$	Boucle principale en n , appel de <code>isInCenter(...)</code> en temps constant, puis coût de mobilité via <code>getWhiteValidMoves()</code> et <code>getBlackValidMoves()</code> .
<code>MaxEvaluator</code>	<code>isInCenter(int, int, int)</code>	$O(1)$	Seulement des bornes calculées et des comparaisons.
<code>IterativeDeepening</code>	<code>calculateBestMove(...)</code>	$\sum_{k=1}^d O(b^k) = O(b^d)$	Boucle sur les profondeurs 1 à d , chaque itération appelant <code>MinMax</code> avec profondeur courante.
<code>MinMax</code>	<code>calculateBestMove(...)</code>	$O(b^d)$	Un appel principal à <code>findBestMoveAtDepth(...)</code> à profondeur <code>maxDepth</code> .
<code>MinMax</code>	<code>findBestMoveAtDepth(...)</code>	$O(b^d)$	Boucle sur les coups racines puis descente récursive dans l'arbre de jeu sans élagage.
<code>MinMax</code>	<code>minMax(...)</code>	$O(b^k)$	Pour profondeur restante k , exploration de tous les coups noirs puis appel récursif <code>maxMin(...)</code> .
<code>MinMax</code>	<code>maxMin(...)</code>	$O(b^k)$	Symétrique de <code>minMax(...)</code> pour les coups blancs, sans coupure.
<code>MinMax</code>	<code>checkTime(long)</code>	$O(1)$	Un seul test booléen via <code>isTimeExceeded(startTime)</code> .
<code>MinMaxAlphaBeta</code>	<code>calculateBestMove(...)</code>	Pire : $O(b^d)$	Lance <code>findBestMoveAtDepth(...)</code> ; l'élagage dépend de l'ordre des coups.
<code>MinMaxAlphaBeta</code>	<code>findBestMoveAtDepth(...)</code>	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Mise à jour de α et β à chaque enfant ; coupures quand $\beta \leq \alpha$.
<code>MinMaxAlphaBeta</code>	<code>maxValue(...)</code>	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Exploration récursive avec arrêt anticipé dès qu'une coupure $\beta \leq \alpha$ est détectée.

Suite à la page suivante...

Classe	Méthode	Complexité	Indice dans le code
MinMaxAlphaBeta	minValue(...)	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Symétrique de maxValuer(...) avec bornes inversées et mêmes règles de coupure.
MinMaxAlphaBeta	checkTime(long)	$O(1)$	Test constant, puis levée d'exception interne si le temps est dépassé.

5.2 Complexité du Board

5.2.1 Hypothèses et notations

Dans cette partie, nous analysons la classe `Board`, qui constitue le moteur de génération et d'application des coups. On note n le nombre de cases actives (32, 50 ou 72 selon la variante), p le nombre de pièces présentes sur le plateau, et c le nombre de pièces capturées par un coup donné. Les opérations bit-à-bit de base (tests, OR, AND, décalages, `bitCount`) sont considérées en temps constant sur mot machine.

5.2.2 Accès, comptage et conversion

Les requêtes élémentaires de présence, comme `isBitWhitePawn(int)`, `isBitBlackPawn(int)`, `isBitWhiteChecker(int)` et `isBitBlackChecker(int)`, reposent sur un test de bit unique ; elles sont donc en $O(1)$. Les vérifications de bord `onLeftEdge(int)` et `onRightEdge(int)` sont également en $O(1)$.

Les conversions d'indices (`indexToSquare(int)` et `boardToBitIndex(int, int)`) sont arithmétiques et valent $O(1)$. La méthode `squareToIndex(String)` reste $O(1)$ du point de vue asymptotique, car la longueur des coordonnées est bornée par la taille du plateau supportée (8, 10, 12).

Les compteurs `whitePawnsCount()`, `blackPawnsCount()`, `whiteCheckersCount()` et `blackCheckersCount()` utilisent `Long.bitCount` sur deux mots 64 bits ; ils s'exécutent donc en $O(1)$.

5.2.3 Application d'un coup et mutations locales

La méthode `applyMove(Move)` comporte trois étapes : déplacement de la pièce, suppression des captures, et promotion éventuelle. Le déplacement et la promotion sont en $O(1)$, tandis que la suppression des captures parcourt la liste `move.getCaptured()` ; le coût est donc $O(c)$. On obtient ainsi :

$$T(\text{applyMove}) = O(1 + c) = O(c).$$

Les helpers de mutation ciblée (`addWhitePawn`, `removeWhitePawn`, `addBlackChecker`, etc.) ne modifient qu'un bit et restent en $O(1)$. Les méthodes utilitaires de test (`restorePiece`, `demoteBit`, `remove`, `addWhite`, `addBlack`, `clearBoard`) sont donc toutes en $O(1)$ dans ce modèle borné.

5.2.4 Génération des coups

La génération globale `getValidMoves(boolean)` effectue un balayage des cases actives puis appelle des routines de coups simples et de prises. Le coût structurel de ce balayage est $O(n)$, mais le coût dominant provient des DFS de captures : `dfsPawn(...)` et `dfsChecker(...)`.

Pour les coups simples, `pawnSimpleTargets(int, boolean)` teste un nombre constant de directions et reste en $O(1)$. `checkerSimpleTargets(int)` parcourt des rayons diagonaux, donc $O(n)$ dans le pire cas (plateau très dégagé).

Pour les prises, `pawnMultiCaptures(...)` et `checkerMultiCaptures(...)` explorent un arbre de séquences de capture. Leur pire cas est exponentiel en profondeur de capture k (en pratique bornée par le plateau), ce que l'on note $O(B^k)$, où B est le facteur de branchement effectif des continuations de prise. Par conséquent, la complexité de `getWhiteValidMoves()` / `getBlackValidMoves()` s'écrit :

$$O(n + \text{coût DFS captures}),$$

et est dominée, dans les positions tactiques, par l'exploration des chaînes de captures.

5.2.5 Copie de plateau

La méthode `copy()` alloue un nouvel objet puis recopie un nombre constant de champs primitifs (huit bitboards + métadonnées). Elle est donc en $O(1)$, ce qui est un point clé pour les algorithmes de recherche.

5.2.6 Synthèse Board

TABLE 4 – Complexité des principales méthodes de Board

Famille	Méthode	Complexité	Justification
Accès bitboard	isBit*, onLeftEdge, onRightEdge	$O(1)$	Un nombre constant de tests/logiques binaires sur un ou deux mots 64 bits.
Conversion	indexToSquare, boardToBitIndex, squareToIndex	$O(1)$	Calculs arithmétiques et parsing de coordonnées de taille bornée.
Mutation locale	add*, remove*, promoteBit, demoteBit, restorePiece	$O(1)$	Modification d'un nombre constant de bits/champs.
Application d'un coup	applyMove (Move)	$O(c)$	Parcours des pièces capturées (taille c), le reste en temps constant.
Coups simples pion	pawnSimpleTargets	$O(1)$	Nombre fixe de directions testées, sans parcours global.
Coups simples dame	checkerSimpleTargets	$O(n)$ (pire cas)	Exploration potentielle de rayons diagonaux longs sur plateau dégagé.
Prises pion/dame	pawnMultiCaptures, checkerMultiCaptures, dfsPawn, dfsChecker	$O(B^k)$ (pire cas)	Exploration récursive d'un arbre de séquences de capture.
Génération globale	getWhiteValidMoves, getBlackValidMoves	$O(n)$ + DFS captures)	Balayage des cases + génération des prises (coût dominant selon la position).
Copie	copy	$O(1)$	Recopie d'un nombre constant de champs primitifs.

6 Explication approfondie

6.1 F36 : Monte Carlo Tree Search (MCTS)

6.1.1 Logique théorique de l'algorithme

Contrairement à l'algorithme Minimax qui tente d'évaluer exhaustivement toutes les branches jusqu'à une profondeur donnée, le MCTS repose sur des simulations statistiques probabilistes. La recherche s'effectue en répétant une boucle de quatre phases fondamentales jusqu'à épuisement du temps de réflexion imparti :

Sélection À chaque itération, l'algorithme part de la racine de l'arbre (l'état actuel du damier) et descend vers les feuilles. Pour choisir quel chemin emprunter, il utilise la formule UCT (Upper Confidence Bound applied to Trees) :

$$UCT_j = \frac{w_j}{n_j} + C \sqrt{\frac{\ln n}{n_j}}$$

Où :

- w_j représente le nombre de victoires accumulées par les simulations passant par le nœud enfant j .
- n_j est le nombre total de simulations ayant traversé ce nœud j .
- n est le nombre total de visites du nœud parent.
- C est la constante d'exploration.

Concrètement : Cette formule résout le dilemme "Exploration vs Exploitation". Le terme de gauche favorise les coups ayant un fort taux de victoire (exploitation), tandis que le terme de droite grandit pour les coups peu testés (exploration). Si un nœud enfant n'a jamais été visité, sa valeur est considérée comme infinie pour forcer l'algorithme à tester chaque coup légal au moins une fois.

Expansion Une fois arrivé sur un nœud dont tous les coups possibles n'ont pas encore été explorés, l'algorithme choisit un de ces mouvements non testés, crée un nouveau nœud dans l'arbre pour représenter ce nouvel état, et passe à la phase suivante.

Simulation Depuis ce nœud nouvellement créé, l'algorithme lance une partie virtuelle. Il joue des coups totalement aléatoires pour les deux joueurs jusqu'à atteindre un état final (victoire, défaite ou égalité).

Rétropropagation Une fois la partie simulée terminée, le résultat (victoire, défaite ou nul) est remonté le long de l'arbre en suivant le chemin exact pris lors de la phase de sélection. Tous les nœuds ancêtres traversés voient leurs statistiques mises à jour : incrémentation de leur compteur de visites, et ajustement de leur score de victoire selon le camp gagnant.

6.1.2 Implémentation et optimisations dans notre projet

Notre classe `Mcts` gère cette recherche de manière optimisée en traduisant cette théorie à travers des choix d'architecture précis.

Traduction des phases en code La boucle principale est gérée par la méthode `calculateBestMove()`, qui s'exécute tant que la limite de temps (`isTimeExceeded()`) n'est pas atteinte.

Lors de la phase de **sélection**, la formule UCT est appliquée par `bestChildByUcb1()`. Si $n_j = 0$, la méthode renvoie `Double.MAX_VALUE` pour assurer l'exploration initiale :

```
double score = child.visits == 0
    ? Double.MAX_VALUE
    : child.winRate() + explorationConstant * Math.sqrt(logVisits / child.visits);
```

Lors de la **simulation** (`simulate()`), pour éviter des parties virtuelles infinies sur des situations bloquées, nous avons introduit une limite stricte de profondeur (`SIMULATION_DEPTH = 60`). Si cette limite est atteinte sans vainqueur, la partie rapporte un score de parité (`DRAW_SCORE = 0.5`), contre `WIN_SCORE = 1.0` pour une victoire.

Optimisation mémoire (ManagerUndoRedo) L'un des plus grands défis du MCTS est la gestion de la mémoire. Au lieu de cloner l'objet `Board` des milliers de fois pour chaque nœud et simulation (ce qui saturerait la RAM et ralentirait drastiquement l'exécution), nous utilisons une instance unique du plateau couplée à un `ManagerUndoRedo`. Chaque coup testé lors de la sélection, de l'expansion et de la simulation est appliqué physiquement sur le plateau, puis proprement annulé lors de la phase de `backpropagate` :

```
// Rétropropagation : on remonte jusqu'à la racine en annulant les coups joués
while (current != null) {
    current.visits++;
    // Mise à jour des scores...

    // Annulation physique du coup sur le plateau unique
    if (current.move != null) {
        undo.undo(current.player == PlayerColor.WHITE);
    }
    current = current.parent;
}
```

Guidage par Machine Learning

Pipeline d'entraînement du modèle Pour générer les poids (`mlWeights`) et le biais (`mlBias`) utilisés par cette fonction, nous avons implémenté un pipeline d'apprentissage supervisé autonome dans la classe `LogisticRegressionTrain`. L'entraînement s'effectue via l'option en ligne de commande `-tr`. Le processus se divise en trois grandes étapes :

1. **Génération des données par Self-Play** : L'application fait s'affronter deux instances de l'IA (MCTS pur) l'une contre l'autre. Pour éviter les parties infinies, une limite stricte de 100 coups est fixée (`MAX_MOVES`). À chaque demi-coup, un vecteur de 6 caractéristiques (*features*) est extrait du plateau :
 - Nombre de pions blancs et noirs.
 - Nombre de dames blanches et noires.
 - Différentiel de pions (Blancs - Noirs).
 - Différentiel de dames (Blancs - Noirs).Si la partie aboutit à une victoire, tous les vecteurs de cette partie sont étiquetés avec le résultat réel (1.0 pour une victoire des blancs, 0.0 pour les noirs). Les parties nulles sont ignorées pour ne pas bruite le modèle.
2. **Optimisation par descente de gradient** : Une fois les données collectées (par défaut sur 10 parties), la méthode `train()` ajuste les poids itérativement sur 1000 *epochs*. À chaque étape, pour chaque vecteur d'état, le modèle effectue une prédiction. L'erreur par rapport au résultat réel (*label*) est calculée, et les poids sont mis à jour selon un taux d'apprentissage (`learningRate = 0.01`) :

$$W_j = W_j - \alpha \times \text{erreur} \times X_j$$

Où α est le taux d'apprentissage, W_j le poids de la caractéristique j , et X_j la valeur de la caractéristique.

3. **Sauvegarde et chargement** : À l'issue de l'entraînement, les poids finaux et le biais sont sauvegardés dans un fichier texte (`ml_weights.txt`). Ce fichier est ensuite chargé dynamiquement par `Mcts` lors de son instantiation, garantissant une séparation propre entre la phase d'entraînement et la phase d'inférence (jeu).

Bilan de l'approche Bien évidemment, ce mode de sélection par apprentissage automatique n'a pas démontré de gain significatif en termes de performance ou de qualité de jeu par rapport à l'UCT classique. Cela s'explique par la simplicité de notre modèle : la régression logistique est un modèle linéaire, et nos 6 *features* mathématiques (uniquement basées sur le comptage matériel) ne capturent pas les subtilités positionnelles du jeu de dames.

Cependant, cette implémentation robuste démontre l'extensibilité de notre architecture. Le pipeline de génération de données, d'entraînement et d'inférence est pleinement fonctionnel, ce qui ouvre la porte à des améliorations futures plus sophistiquées (comme le remplacement de la régression logistique par un réseau de neurones profond évaluant des motifs complexes) pour renforcer drastiquement l'intelligence de notre agent.

Détermination du meilleur coup final À l'épuisement du temps de réflexion imparti, l'arbre cesse de croître. Quelle que soit l'heuristique de sélection utilisée durant la recherche (la formule UCT classique ou les prédictions du modèle de Machine Learning), la décision finale ne se base pas sur ces scores d'évaluation.

L'IA fait appel à la méthode `mostVisitedChild()`, qui sélectionne le nœud enfant de la racine possédant le **plus grand nombre de visites**. Ce critère est privilégié car il est statistiquement le plus robuste : une branche très visitée signifie qu'elle a continuellement été considérée comme la plus prometteuse par l'algorithme, et qu'elle a survécu à un grand nombre de simulations face aux meilleures réponses possibles de l'adversaire.

Stratégie de validation et de tests Afin de garantir la robustesse de notre implémentation du MCTS, nous avons développé une série de tests unitaires centralisés dans la classe `MctsTest`. La première étape de cette validation consiste à vérifier l'intégrité de l'arbre de recherche et de son cycle de vie. Cela passe notamment par le contrôle de l'initialisation des paramètres fondamentaux, comme l'affectation correcte de la constante d'exploration à sa valeur théorique par défaut de $\sqrt{2}$. Nous nous assurons également de la stabilité de l'algorithme face aux cas limites. Par exemple, si l'IA est sollicitée sur un plateau où aucun mouvement légal n'est possible (en toute fin de partie), la méthode de recherche est tenue de retourner une absence de coup sans provoquer la moindre erreur d'exécution.

```
@DisplayName("Default constructor sets exploration constant to sqrt(2)")
void testDefaultConstructorExplorationConstant() {
    Mcts mcts = new Mcts();
    assertEquals(Mcts.DEFAULT_EXPLORATION, mcts.explorationConstant, 1e-9);
}
```

Le mode de sélection par apprentissage automatique (`SelectionMode.ML`) a nécessité une approche de test plus pointue. Étant donné que cette fonctionnalité manipule des classes internes privées, comme les nœuds de l'arbre, et lit des fichiers de poids externes, nous avons fait appel à la réflexion Java pour isoler et éprouver ces composants. Cette technique nous permet de confirmer que la fonction de chargement analyse correctement les vecteurs de caractéristiques, tout en gérant proprement les fichiers corrompus via des exceptions contrôlées. De plus, la réflexion nous sert à simuler artificiellement des nœuds en cours de partie pour vérifier que l'algorithme bascule bien sur le moteur d'inférence ML, au lieu de la formule UCT, lorsque l'option correspondante est activée au lancement.

```
@DisplayName("Test direct du switch case ML dans select")
void testSelectOnlyMl() throws Exception {
    mcts.setSelectionMode(SelectionMode.ML);
    Object result = selectMethod.invoke(mcts, root, board, undo);
    assertEquals(child, result, "Le switch doit choisir l'enfant via le mode ML");
}
```

Enfin, nous contrôlons rigoureusement la cohérence mathématique des calculs qui guident les choix de l'IA. Nos tests s'assurent par exemple que l'évaluation par régression logistique génère systématiquement une probabilité valide, strictement bornée entre 0 et 1, grâce à l'application de la fonction sigmoïde sur les statistiques du plateau. Du côté de l'approche statistique classique, nous validons le mécanisme de convergence de l'UCT en vérifiant que le code attribue bien la priorité maximale (`Double.MAX_VALUE`) aux branches non explorées. C'est cette garantie mathématique qui force l'algorithme à évaluer chaque coup légal au moins une fois avant de se concentrer sur l'exploitation des meilleures variantes.

```
@DisplayName("evaluateMl correctly calculates probability with loaded weights")
void testEvaluateMlCalculation(@TempDir Path tempDir) throws Exception {
    double probability = (double) evaluateMlMethod.invoke(mcts, childNode, board, undo);
    assertTrue(probability >= 0.0 && probability <= 1.0);
}
```

F39 : Intégration du réseau, des interfaces et de la synchronisation

6.1.3 Objectif général de la fonctionnalité

La fonctionnalité F39 correspond à l'intégration complète de la couche réseau dans notre application de jeu de dames. Son objectif n'était pas uniquement de permettre à deux machines d'échanger des messages via TCP, mais de faire en sorte que cette communication distante s'intègre proprement à l'architecture existante, aux échanges de coup aussi bien du côté de l'interface en ligne de commande (CLI) que du côté de l'interface graphique (GUI). Concrètement, il fallait que le même moteur de jeu puisse être piloté localement ou à distance, sans dupliquer la logique métier, tout en ajoutant les commandes nécessaires à l'administration du serveur, à la création de parties réseau et à la consultation de l'état global du système. Cette fonctionnalité relie donc plusieurs composants centraux du projet : `ClientSession`, `GameServer`, `GameRegistry`, `GameSession` et `PlayerSession`.

6.1.4 Articulation entre mode local, mode serveur et mode connecté

Pour rendre cette intégration cohérente, nous avons d'abord introduit une notion explicite de **mode d'exécution** grâce à l'énumération `ClientMode`. Trois états ont été définis : `LOCAL`, lorsque l'application fonctionne sans connexion réseau ; `SERVER`, lorsqu'un serveur est démarré localement ; et `CONNECTED`, lorsqu'un client est relié à un serveur distant. Cette séparation est essentielle, car elle permet de contrôler quelles commandes sont autorisées selon le contexte. Par exemple, un client déjà connecté ne doit pas pouvoir démarrer un serveur local, tandis qu'une instance qui héberge déjà un serveur ne doit pas réutiliser certaines commandes purement clientes. Cette logique de modes structure donc toute la navigation de l'utilisateur dans l'application et évite les conflits d'usage entre les différents scénarios d'exécution. Les transitions entre ces états sont gérées dans `ClientSession` via des méthodes comme `enterServerMode()`, `exitServerMode()` et `getMode()`.

6.1.5 Articulation entre mode local, mode serveur et mode connecté

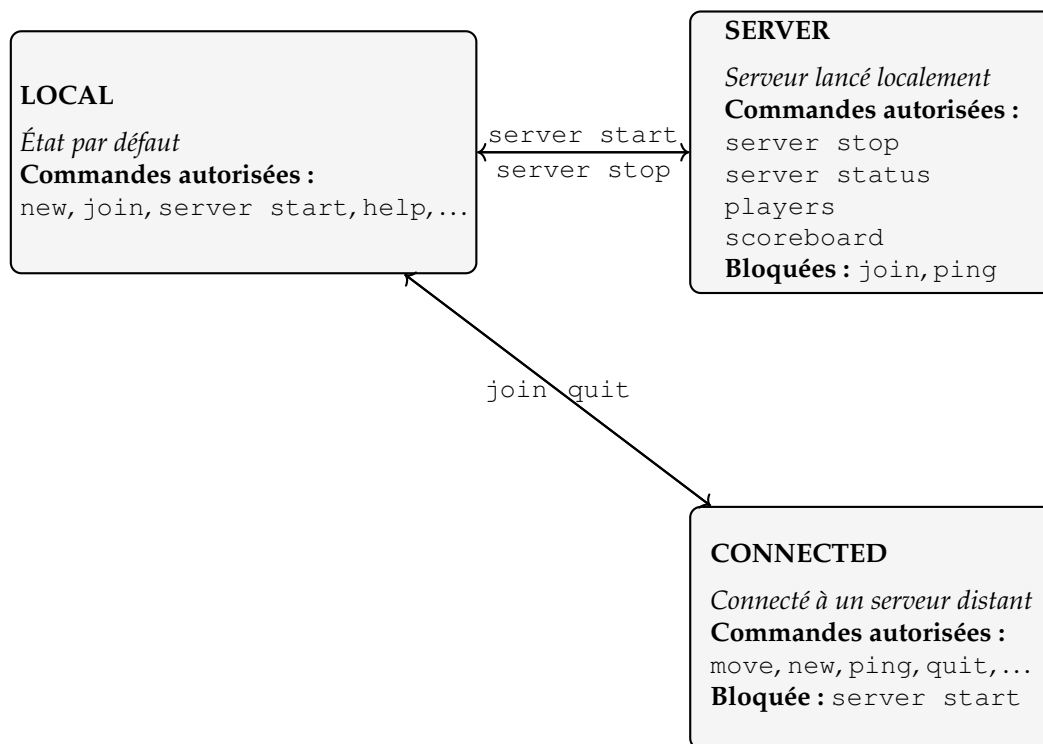


FIGURE 15 – Modes d'exécution du client et commandes autorisées selon `ClientMode`.

Trois états ont été définis : `LOCAL`, lorsque l'application fonctionne sans connexion réseau ; `SERVER`, lorsqu'un serveur est démarré localement ; et `CONNECTED`, lorsqu'un client est relié à un serveur distant. Cette séparation est essentielle, car elle permet de contrôler quelles commandes sont autorisées selon le mode courant du jeu. Par exemple, un client déjà connecté ne doit pas pouvoir démarrer un serveur alors qu'il est déjà engagé dans une session distante, tandis qu'une instance qui héberge déjà un serveur ne doit pas réutiliser certaines commandes purement clientes comme `join`. Cette logique de modes structure donc toute la navigation de l'utilisateur dans l'application et évite les conflits d'usage entre les différents scénarios d'exécution. Les transitions entre ces états sont gérées dans `ClientSession` via des méthodes comme `enterServerMode()`, `exitServerMode()` et `getMode()`.

6.1.6 Connexion réseau et intégration avec les interfaces CLI et GUI

La classe `ClientSession` constitue le point d'entrée principal de la logique réseau côté client. Elle encapsule l'ensemble de l'état de connexion : le socket TCP (`socket`), les flux d'entrée et de sortie (`in` et `out`), l'état booléen `connected`, ainsi que l'adresse du serveur courant dans `currentServer`. Elle fournit les opérations fondamentales `connect()`, `disconnect()` et `send()`, qui permettent respectivement d'établir la connexion, de la fermer proprement et d'envoyer des commandes texte au serveur. Une fois connecté, le client lance un thread dédié par `startListenerThread()`, chargé d'écouter en permanence les messages provenant du serveur. Cette architecture permet au client de rester interactif pendant que les mises à jour réseau arrivent en arrière-plan.

L'intérêt principal de cette conception est qu'elle unifie le comportement entre le mode CLI et le mode GUI. En effet, les messages reçus du serveur, comme `GAME_START`, `MOVE_OK` ou `OPPONENT_MOVE`, sont tous interprétés dans `handleServerMessage()` puis réinjectés dans le même `GameController` local via `executeMove()`. En mode CLI, cela se traduit par un affichage textuel immédiat dans le terminal ; en mode GUI, la même mise à jour du contrôleur entraîne naturellement le rafraîchissement de l'affichage graphique. Ainsi, le projet ne possède pas un moteur pour le terminal et un autre pour l'interface graphique : il possède un contrôleur unique, alimenté soit par des actions locales, soit par des événements réseau. Cette mutualisation garantit la cohérence fonctionnelle entre les deux interfaces et limite fortement la duplication de code.

6.1.7 Architecture serveur et isolation des sessions de jeu

Côté serveur, la classe `GameServer` assure la gestion des connexions entrantes, la création des parties et la supervision globale du service. Elle ouvre un `ServerSocket` sur un port TCP donné, puis accepte les connexions de plusieurs clients successifs. Chaque client est pris en charge dans un thread distinct grâce à un pool créé par `Executors.newCachedThreadPool()`, ce qui permet au serveur de traiter plusieurs joueurs simultanément. Lorsqu'un client s'enregistre, il est représenté sous la forme d'un objet `PlayerSession` conservé dans `GameRegistry`, qui joue le rôle de registre partagé des joueurs connectés et des parties actives. Lorsqu'une partie est créée, le serveur construit un nouveau `GameController` par l'intermédiaire de `GameControllerFactory`. Cela garantit qu'une partie réseau possède son propre état de jeu, totalement indépendant des autres. Deux parties simultanées ne partagent donc ni le plateau, ni l'historique, ni le tour courant. Cette isolation était indispensable pour rendre possible un véritable serveur multi-clients.

La classe `GameSession` représente précisément une partie active. Elle centralise la logique d'une session multijoueur : ordre des tours, validation des coups, diffusion des notifications `OPPONENT_MOVE` et terminaison de la partie. La méthode `handleMove()` vérifie qu'un joueur agit bien au bon moment, applique le coup via le contrôleur et en informe les autres participants. La méthode `end()` clôt ensuite la session en mettant à jour les statistiques et en rétablissant les joueurs dans un état disponible. Cette structuration permet d'encapsuler clairement ce qui relève d'une partie particulière, sans polluer le serveur global avec des détails de jeu.

6.1.8 Commandes réseau et commandes d'administration ajoutées

F39 s'est également traduite par l'ajout ou la consolidation de plusieurs commandes réseau. Dans le flux de communication client-serveur, des commandes comme `REGISTER`, `PING`, `STATUS`, `PLAYERS`, `NEW`, `MOVE` ou `QUIT` permettent respectivement d'enregistrer un joueur, mesurer la connectivité, consulter l'état du serveur, lister les joueurs présents, créer une nouvelle partie, envoyer un coup et fermer proprement la connexion. Ce protocole texte simple présente un double avantage : il est facile à déboguer et il reste compatible avec une interface CLI tout en étant suffisamment structuré pour piloter une interface graphique. Côté client, `ClientSession` assure l'émission et l'interprétation de ces messages ; côté serveur, `GameServer` les décode pour déclencher les actions appropriées.

À cela s'ajoutent des commandes de gestion et d'observation du serveur. La commande `server status`, implémentée dans `ServerStatusCommand`, affiche le port d'écoute ainsi que le nombre de clients et de parties actives. La commande `scoreboard`, implémentée dans `ScoreboardCommand`, classe les joueurs selon leur nombre de victoires et affiche leurs statistiques globales. Ces commandes montrent que le réseau n'a pas été conçu uniquement comme un canal de transport des coups du jeu, mais aussi comme une infrastructure administrable, observable et exploitable en cours d'exécution. Elles participent donc pleinement à la valeur de F39, car elles rendent la couche réseau utilisable au quotidien, aussi bien pour jouer que pour surveiller le système.

6.1.9 Cohérence avec les options de lancement et la commande new

L'intégration réseau a également eu des conséquences sur le point d'entrée de l'application et sur la gestion des options. Dans notre architecture, le lancement du programme doit pouvoir distinguer plusieurs scénarios : démarrage en mode local, ouverture d'un serveur, connexion à un serveur distant, ou encore démarrage d'une partie paramétrée avec des options avancées. Cette cohérence est visible à travers la commande `new`, implémentée par `NewCommand`, qui permet déjà de configurer de manière fine une partie locale : mode blitz, taille du plateau, IA activées, temps de réflexion, profondeur Minimax, mode de sélection du MCTS et fonction d'évaluation. L'ajout du réseau devait donc rester compatible avec cette philosophie générale : le système ne devait pas opposer brutalement "jeu local"

et “jeu réseau”, mais prolonger la même logique de configuration dans de nouveaux contextes d’exécution. C’est dans ce cadre que les nouvelles options ajoutées au lancement via `App.java` prennent tout leur sens : elles servent à choisir explicitement le mode d’utilisation de l’application, tout en conservant une cohérence avec les commandes déjà existantes dans le shell. Autrement dit, F39 ne modifie pas seulement le transport des données ; elle étend aussi la manière dont l’application est initialisée et pilotée par l’utilisateur.

6.1.10 Gestion de la concurrence et apport des mutex

L’un des aspects les plus techniques de F39 concerne la gestion de la concurrence. L’introduction du réseau fait apparaître plusieurs flux d’exécution simultanés. Côté client, on retrouve au minimum le thread principal, qui lit les commandes utilisateur, et le thread secondaire lancé par `startListenerThread()`, qui écoute les messages entrants du serveur. Côté serveur, chaque connexion cliente est traitée dans un thread potentiellement distinct, ce qui implique des accès concurrents aux mêmes structures globales. Sans protection, plusieurs problèmes peuvent apparaître : lecture incohérente de l’état `connected`, écriture sur un socket en cours de fermeture, double création de partie avec les mêmes joueurs, ou mise à jour concurrente des statistiques. C’est précisément pour corriger ces risques que nous avons introduit des **mutex** et des protections explicites sur les sections critiques.

Dans la version améliorée de cette fonctionnalité, l’état de connexion du client a été protégé par un verrou dédié, afin de sérialiser les accès à `socket`, `in`, `out`, `connected` et `currentServer`. Ce verrou évite qu’un thread d’écoute ferme ou lise la connexion pendant qu’un autre thread tente simultanément d’envoyer un message ou d’interroger l’état courant de la session. Du côté serveur, nous avons renforcé la sûreté d’exécution en protégeant les structures partagées et les opérations de cycle de vie : le registre central `GameRegistry` doit manipuler les collections de joueurs et de sessions de manière atomique, tandis que la logique des parties dans `GameSession` doit empêcher qu’un même tour soit joué deux fois en parallèle. De même, `PlayerSession` centralise l’état individuel des joueurs et leurs statistiques ; ces données doivent donc rester cohérentes malgré des accès depuis plusieurs threads. Le rôle des mutex n’est donc pas d’ajouter une nouvelle fonctionnalité visible, mais de garantir que toutes les fonctionnalités réseau précédemment décrites restent correctes, même en environnement multi-thread.

6.1.11 Découverte réseau et ergonomie d’utilisation

Enfin, F39 comprend aussi un aspect ergonomique avec les services de découverte réseau. La classe `DiscoveryService` diffuse régulièrement la présence du serveur sur le réseau local via UDP, tandis que `ServerListService` écoute ces annonces et construit une liste des serveurs actifs encore disponibles. Cette fonctionnalité améliore l’expérience utilisateur, car elle évite d’avoir à saisir manuellement toutes les informations de connexion. Elle montre également que l’intégration réseau a été pensée dans sa globalité : non seulement le système permet de jouer à distance, mais il aide aussi à découvrir automatiquement les serveurs présents dans l’environnement local.

6.1.12 Bilan de la fonctionnalité

En définitive, F39 représente une fonctionnalité transversale majeure du projet. Elle ne se réduit ni à l’ouverture d’un socket, ni à l’affichage d’un plateau graphique connecté. Elle articule la communication réseau, la compatibilité entre CLI et GUI, l’ajout de nouvelles commandes de jeu et d’administration, l’intégration avec les options de lancement, la création de sessions indépendantes et la sécurisation des accès concurrents par mutex et synchronisation. Cette fonctionnalité illustre donc un véritable travail d’architecture logicielle : nous avons transformé un moteur de jeu initialement local en un système multi-clients cohérent, extensible et robuste, capable de fonctionner correctement dans un environnement distribué et concurrent tout en restant fidèle à l’organisation générale du projet.

7 Revue critique de projet

7.1 Bug

7.2 Axes d’amélioration

Conclusion

Références

- [1] Brandon Corfman. Raven Checkers — open-source checkers with ai. <https://github.com/bcorfman/raven-checkers>, 2023. Consulté le 8 avril 2026.
- [2] mortexsa. AwsDame — multiplayer checkers platform. <https://github.com/mortexsa/AwsDame>, 2023. Consulté le 8 avril 2026.
- [3] Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson, 4 edition, 2020.

A Liste des 40 besoins

TABLE 5 – État d’avancement des besoins fonctionnels

ID	Description du besoin	Statut
F1	Lancement du programme	
F2	Fichier de configuration	
F3	Internationalisation (i18n)	
F4	Interface en ligne de commande	
F5	Mode blitz	
F6	Mode contest	
F7	Interface graphique	
F8	Mode IA	
F9	Taille du damier	
F10	Gestion des règles	
F11	Gestion des erreurs de l'utilisateur	
F12	Gestion du undo/redo	
F13	Gestion des conseils	
F14	Gestion du blitz	
F15	Shell interactif	
F16	Proposer de sauvegarder avant de quitter	
F17	Édition de la commande en cours	
F18	Historique des commandes tapées	
F19	Complétion des commandes	
F20	Notation des coups	
F21	Sauvegarde et restauration d'une partie	
F22	Commentaires dans la sauvegarde	
F23	Format des paramètres du jeu	
F24	Format du damier	
F25	Format de l'historique	
F26	Module bitboard	
F27	Algorithmes de manipulation des bitboards	
F28	Menus de l'application	
F29	Raccourcis clavier	
F30	Drag'n'Drop	
F31	Temps de réflexion borné	
F32	Minimax avec $\alpha\beta$ -pruning	
F33	Fonctions d'évaluation pour Minimax	
F34	Approfondissement itératif de Minimax	
F35	Profondeur de recherche de Minimax	
F36	Monte Carlo Tree Search (MCTS)	
F37	Fonction de sélection par Machine Learning	
F38	Découverte de serveurs et connexion client-serveur basique	
F39	Serveur multi-clients et gestion de parties	
F40	Système d'invitations et gestion avancée des joueurs	